

DevOps. Automatización y monitorización de infraestructuras

Unidad 02. Introducción a la automatización con Vagrant - Laboratorio 01



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 02. Introducción a la automatización con Vagrant - Laboratorio 01	2
1. Explicación ¿Que haremos en este caso práctico?	2
2. Objetivos	2
3. El modelo mental	2
4. Topología del proyecto	3
5. Fase de ejecución: despliegue paso a paso	3
Paso 1: Orquestación de red (El Cimiento)	3
Paso 2: Construcción del laboratorio (Vagrant)	4
Paso 3: Auditoría Automatizada (Verificación)	7
6. Pruebas manuales	7
7. Operaciones de limpieza	8
8. Problemas comunes	9

UNIDAD 02. INTRODUCCIÓN A LA AUTOMATIZACIÓN CON VAGRANT - LABORATORIO 01

1. EXPLICACIÓN ¿QUE HAREMOS EN ESTE CASO PRÁCTICO?

En este laboratorio vamos a construir una infraestructura basada en **IaC (Infrastructure as Code)** desde su nivel más atómico.

La idea es preparar un entorno para el alumnado automatizado, sin instaladores manuales ni necesidad de interfaces gráficas. En su lugar, orquestaremos la creación de una red *bridge* directamente en el Host y desplegamos dos servidores Debian (un servidor web y un cliente de pruebas) de forma totalmente automatizada.

2. OBJETIVOS

Este no es un simple tutorial de "siguiente, siguiente, finalizar". Al terminar este caso práctico, habrás logrado:

- **Aislamiento:** Desplegar una red de laboratorio aislada dedicada (**LabCEFire**) mediante scripts.
- **Inmutabilidad:** Automatizar la creación de servidores con Vagrant que sean idénticos cada vez que los levantes.
- **Orquestación:** Entender la jerarquía de dependencias en la provisión modular de servicios (primero red, luego herramientas base, finalmente Nginx).
- **Diagnóstico:** Realizar pruebas forenses de conectividad entre el Host y la máquina virtualizada (Guest).

3. EL MODELO MENTAL

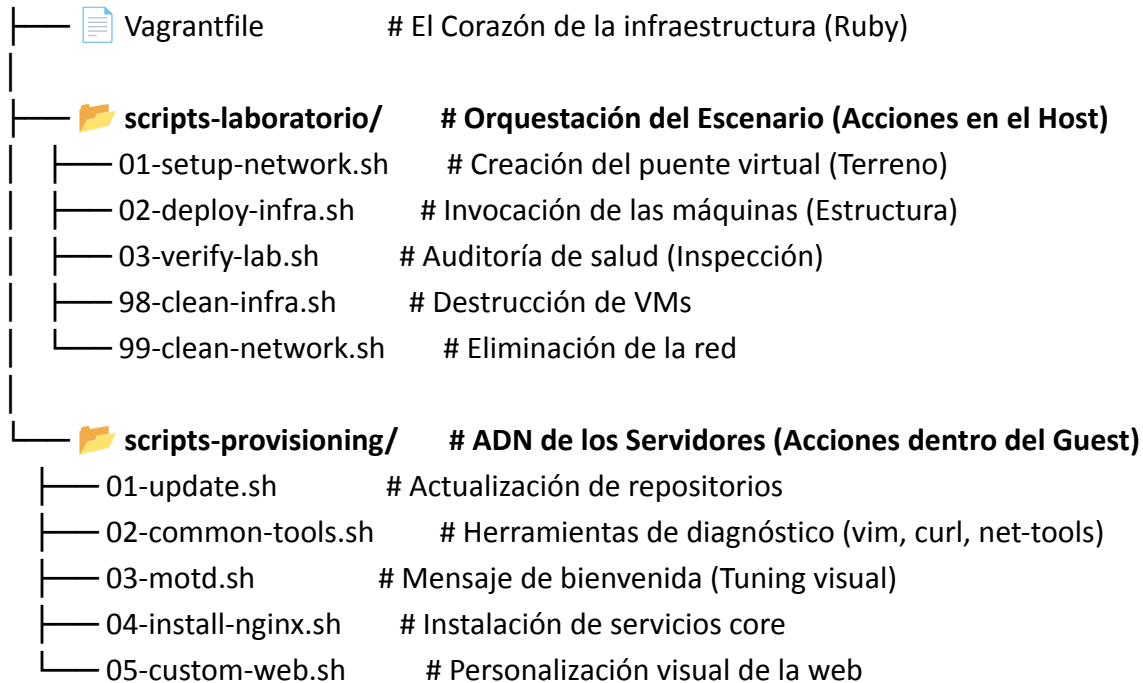
Para entender cómo encajan todas las piezas de este laboratorio, imagina que vas a montar una tienda de campaña de alta tecnología en medio del bosque:

1. **El terreno (Red Bridge):** Primero buscas un terreno llano y pones una lona aislante en el suelo. Esto es crear el puente virtual en tu PC para que el entorno esté aislado de la red del aula o de tu casa.
2. **La estructura (Vagrant Up):** Luego clavas las piquetas y levantas la estructura metálica. Aquí es donde Vagrant reserva la RAM y el disco duro.
3. **El interior (Provisión modular):** Los operarios entran y montan las camas, la cocina y las luces. Estos son nuestros scripts ejecutándose secuencialmente para instalar Nginx y configurar el entorno.
4. **La inspección (Verificación):** Finalmente, compruebas que la cremallera de la puerta abre y que la luz enciende. Es nuestra auditoría forense con curl y ping.

4. TOPOLOGÍA DEL PROYECTO

Para mantener la cordura cuando gestionamos infraestructura como código, el orden de los archivos lo es todo. Esta es la estructura que vamos a utilizar:

Unidad-02-Vagrant/Laboratorio/



5. FASE DE EJECUCIÓN: DESPLIEGUE PASO A PASO

Antes de poner las manos en el teclado, asegúrate de cumplir con los **requisitos previos** para no encontrarte con errores de hardware:

- **Vagrant v2.3+ y VirtualBox v7.0+.**
- **Sistema Host Linux.**
- **Mínimo 2GB de RAM física** libres en tu equipo.

Abre tu terminal y sitúate en el centro de mando (la carpeta raíz del proyecto):

```
cd Unidad-02-Vagrant/Laboratorio/
```

Paso 1: Orquestación de red (El Cimiento)

Ejecutamos el script que crea la "carretera" por la que viajarán los datos antes siquiera de encender una máquina:

```
bash scripts-laboratorio/01-setup-network.sh
```

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-02-Vagrant/Laboratorio$ bash scripts-laboratorio/01-setup-network
.sh
UD02 - Configuración de Red Bridge (Host) |
[RED] Creando bridge 'LabCEFIRE'...
[RED] Forzando estado UP con interfaz dummy...
✓ Red configurada con éxito.
IP del Host (Gateway): 172.28.0.1
Estado: UP
```

- **¿Qué ocurre bajo el capó?** El script no usa Vagrant, habla directamente con el Kernel de Linux del Host.
- **Comandos clave del script:**
 - ***modprobe dummy***: Carga el módulo para interfaces de red virtuales.
 - ***ip link add name LabCEFIRE type bridge***: Crea un switch lógico totalmente aislado de tu red WiFi/Ethernet real.
 - ***ip addr add 172.28.0.1/24 dev LabCEFIRE***: Le asigna a tu PC anfitrión (el Host) la IP que actuará como puerta de enlace de esa red.

Paso 2: Construcción del laboratorio (Vagrant)

Lanzamos el orquestador para que la infraestructura cobre vida:

```
bash scripts-laboratorio/02-deploy-infra.sh
```

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-02-Vagrant/Laboratorio$ bash scripts-laboratorio/02-deploy-infra.
sh
UD02 - FASE 2: Despliegue de Infraestructura ||
[VAGRANT] Levantando máquinas virtuales...
Este proceso puede tardar unos minutos (descarga de box y provisión)...
Bringing machine 'webserver' up with 'virtualbox' provider...
Bringing machine 'client' up with 'virtualbox' provider...
==> webserver: Importing base box 'debian/bookworm64'...
Progress: 90%█
```

- **¿Qué ocurre bajo el capó?** Este script es un envoltorio que lanza el comando maestro ***vagrant up***.
- **Comandos y procesos clave:**
 - Vagrant lee el ***Vagrantfile***.
 - Hace un *Copy-on-Write* de la Box inmutable de Debian.
 - Llama a la API de VirtualBox (***VBoxManage***) para reservar la RAM exacta (1024MB y 512MB) y habilitar el modo promiscuo en las tarjetas de red.
 - Se inyecta por SSH y ejecuta de forma secuencial los scripts 01 al 05 (actualizando repositorios, instalando herramientas e instalando Nginx).

2.1. El Plano Maestro: Diseccionando el Vagrantfile (Deep Dive)

Justo cuando lanzas **02-deploy-infra.sh**, el motor de Vagrant busca inmediatamente el archivo Vagrantfile. Este archivo Ruby es la única fuente de verdad. Si no está aquí, no existe. Estas son las directivas críticas que Vagrant traduce en llamadas al hipervisor (VirtualBox):

- **La Fundación (`config.vm.box = "debian/bookworm64"`):**
 - **La explicación del arquitecto:** Le dice a Vagrant qué imagen inmutable debe usar como cimiento. Al elegir Debian 12 (Bookworm), garantizamos un entorno estable y ligero.
- **Topología Multi-Nodo (`config.vm.define "webserver" do |web|`):**
 - **La explicación del arquitecto:** En lugar de una sola máquina, iteramos el despliegue creando un bloque lógico para el webserver y otro para el client. Esto permite aislar variables y asignar recursos de forma asimétrica.
- **El enlace físico (`web.vm.network "public_network", ip: "172.28.0.20", bridge: "LabCE FIRE"`):**
 - **La explicación del arquitecto:** Esta es la magia del laboratorio. Le indicamos a Vagrant que no use su red NAT por defecto, sino que conecte la tarjeta de red virtual directamente al switch lógico (**LabCE FIRE**) que creamos previamente en el Host, asignándole una IP estática inamovible.
- **Asignación de memoria (`vb.memory = "1024"`):**
 - **La explicación del arquitecto:** Llama a la API de VirtualBox para que reserve bloques reales de memoria RAM en tu placa base. Le damos 1GB al servidor y solo 512MB al cliente de pruebas, optimizando nuestros recursos.
- **La trampilla del Hipervisor (`vb.customize ["modifyvm", :id, "--nicpromisc2", "allow-all"]`):**
 - **La explicación del arquitecto:** Este es el comando de más bajo nivel del archivo. A veces Vagrant no tiene un atajo amigable para funciones avanzadas de red. Esta directiva inyecta un comando "crudo" de **VBoxManage** directamente al motor de VirtualBox para habilitar el Modo Promiscuo. Sin esto, las tarjetas virtuales rechazarían los paquetes del Bridge y las máquinas quedarían incomunicadas.
- **El disparador de provisión (`web.vm.provision "shell", path: "..."`):**
 - **La explicación del arquitecto:** Una vez que el hardware virtual está ensamblado y el kernel de Debian arranca, estas líneas actúan como un gatillo. Le dicen a Vagrant qué scripts externos debe inyectar por SSH y en qué orden estricto debe ejecutarlos.

2.2. El aprovisionamiento: anatomía de los scripts

Para entender a nivel arquitectónico qué pasa durante el **vagrant up**, debemos hacer zoom al momento exacto en el que VirtualBox termina de ensamblar el hardware virtual (la RAM, la CPU, la tarjeta de red). En ese instante, Vagrant inyecta una clave privada, abre una sesión SSH contra el sistema recién nacido y comienza a ejecutar el Aprovisionamiento Modular.

No inyectamos un script gigante, sino que usamos "piezas de Lego" secuenciales. Aquí tienes el

desglose forense de lo que ocurre dentro del sistema operativo invitado (Guest OS):

Módulo 1: Preparación del Sistema (01-update.sh)

- Comando crítico: ***apt-get update -qq***
- La explicación del arquitecto: Lo primero que necesita un servidor es saber qué software existe en el mundo. Este script actualiza los índices de los repositorios de Debian. El uso del flag -qq (ultra-silencioso) es vital en la Infraestructura como Código: evita que el gestor de paquetes vomite miles de líneas en tu terminal del Host, lo que haría ilegible el log de despliegue.

Módulo 2: Inyección de Herramientas Core (02-common-tools.sh)

- Comandos críticos:
 - ***export DEBIAN_FRONTEND=noninteractive***
 - ***apt-get install -y -qq vim curl git htop net-tools > /dev/null***
- La explicación del arquitecto: Aquí instalamos nuestro cinturón de herramientas (editores, analizadores de red y monitores de procesos). La variable de entorno ***DEBIAN_FRONTEND=noninteractive*** es el escudo definitivo: le prohíbe al sistema operativo lanzar ventanas de diálogo azules pidiendo confirmación humana. Sumado al flag -y (responder "Sí" a todo), garantizamos que la instalación no se quede bloqueada esperando a que alguien pulse "Enter". Todo el output se envía al agujero negro > /dev/null para mantener la limpieza.

Módulo 3: Identidad y Tuning Visual (03-motd.sh)

- Comando crítico: Uso de inyección directa con ***cat <<EOF> /etc/motd***
- La explicación del arquitecto: El Message Of The Day (MOTD) es lo primero que ves al entrar por SSH. En lugar del texto estándar aburrido, inyectamos variables dinámicas del Kernel como ***\$(hostname)*** y ***\$(hostname -I)***. Así, cuando hagamos un vagrant ssh, veremos un panel de control nítido confirmando en qué máquina y en qué IP exacta acabamos de aterrizar.

Módulo 4: Despliegue del Servicio HTTP (04-install-nginx.sh) (Solo en el nodo Webserver)

- Comando crítico: ***apt-get install -y -qq nginx > /dev/null***
- La explicación del arquitecto: Este script transforma un simple servidor en un servidor web. Al instalar el paquete ***nginx*** de forma desatendida, el gestor de demonios de Debian (systemd) asume el control y levanta automáticamente el proceso HTTP en el puerto 80 sin que tengamos que ordenarle un systemctl start explícitamente.

Módulo 5: Personalización de la Capa de Aplicación (05-custom-web.sh) (Solo en el nodo Webserver)

- La explicación del arquitecto: Reemplaza la página estandarizada de bienvenida de Nginx (***/var/www/html/index.html***) por el código HTML personalizado del laboratorio.

 La Jerarquía de dodos del Vagrantfile es lo suficientemente inteligente como para no ejecutar todo en todas partes:

- El servidor web (***ud02-webserver***) ejecuta la secuencia completa: scripts del 01 al 05.

- El cliente de pruebas (*ud02-client*) solo ejecuta los scripts del 01 al 03. Al ser una simple estación de diagnóstico de red, no necesita el peso de tener un motor Nginx ni archivos HTML instalados, ahorrando recursos y tiempo de despliegue.

Paso 3: Auditoría Automatizada (Verificación)

Comprobamos que las piezas han encajado correctamente:

```
bash scripts-laboratorio/03-verify-lab.sh
```

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-02-Vagrant/Laboratorio$ bash scripts-laboratorio/03-verify-lab.sh

UD02 - FASE 3: Verificación del Laboratorio

===== 1. ESTADO DE LAS MÁQUINAS VIRTUALES =====
webserver      running (virtualbox)
client         running (virtualbox)

===== 2. ESTADO DE LA RED (BRIDGE) =====
✓ Red LabCEfire: ACTIVA
  inet 172.28.0.1/24 scope global LabCEfire

===== 3. VERIFICACIÓN DE SERVICIOS (HTTP) =====
✓ Servidor Web (172.28.0.20): RESPONDE

===== 4. PRUEBA DE CONECTIVIDAD (PING) =====
✓ Ping Cliente -> Servidor: EXITOSO

✓ Verificación finalizada con éxito
```

- **¿Qué ocurre bajo el capó?** El script lanza herramientas de diagnóstico para validar el stack de red y los servicios.
- **Comandos clave del script:**
 - **vagrant status:** Pide telemetría al hipervisor para asegurar que las VMs están en estado running.
 - **curl -s -I http://172.28.0.20:** Golpea el puerto 80 del servidor web desde el Host para ver si Nginx responde con un código HTTP 200.
 - **vagrant ssh client -c "ping -c 1 172.28.0.20":** Usa el cliente de pruebas como sonda para asegurar que el enrutamiento interno de Capa 3 entre las dos VMs funciona perfectamente.

6. PRUEBAS MANUALES

Aunque el script **03-verify-lab.sh** hace las comprobaciones por nosotros mediante la terminal, la prueba de fuego en cualquier despliegue web es la experiencia del usuario. Una vez montado el laboratorio, realiza estas tres pruebas:

1. **La prueba del navegador:** Abre Firefox, Chrome o tu navegador favorito en tu PC anfitrión y escribe la IP del servidor: <http://172.28.0.20>.
 - **Objetivo:** Ver la página web modificada. Esto confirma que el script de aprovisionamiento **05-custom-web.sh** se ejecutó con éxito.

2. La Prueba de la Trinchera: Entra dentro del cliente de pruebas ejecutando:

```
vagrant ssh client
```

- Una vez dentro (fíjate que el *prompt* de tu terminal habrá cambiado), comprueba la conectividad interna:

```
ip a # Verifica que La IP asignada es La 172.28.0.21
curl 172.28.0.20 # Descarga La web del servidor desde dentro de La red
exit # Sal de La máquina virtual
```

3. **El radar general:** Ejecuta ***vagrant global-status***. Esto te mostrará el identificador único (UUID) y el estado de todas las máquinas virtuales que Vagrant está gestionando en tu ordenador.

7. 🧼 OPERACIONES DE LIMPIEZA

Un buen arquitecto no deja recursos huérfanos consumiendo RAM o ciclos de CPU en su máquina cuando termina de trabajar. Para destruir el laboratorio y devolver tu PC a su estado original, ejecuta estos dos scripts estrictamente en este orden:

1. Destrucción de las Máquinas:

```
bash scripts-laboratorio/98-clean-infra.sh
```

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-02-Vagrant/Laboratorio$ bash scripts-laboratorio/98-clean-infra.sh
h
  ⚠️ UD02 - DESTRUCCIÓN DE INFRAESTRUCTURA ||
[VAGRANT] Destruyendo máquinas virtuales...
[VAGRANT] Limpiando archivos de estado (.vagrant)...
✓ Entorno de infraestructura limpio.
NOTA: La red 'LabCEFIRE' sigue activa. Usa 99-clean-network.sh para borrarla.
```

- **Comandos clave:** Ejecuta un ***vagrant destroy -f*** (la flag -f evita que te pida confirmación manual). Además, ejecuta ***rm -rf .vagrant/*** para borrar los metadatos de estado, evitando el síndrome de las "máquinas zombie".

2. Demolición del Puente Virtual:

```
bash scripts-laboratorio/99-clean-network.sh
```

```
alumno@alumno-virtualbox:~/Escritorio/Unidad-02-Vagrant/Laboratorio$ bash scripts-laboratorio/99-clean-network.sh
h
  ⚠️ UD02 - ELIMINACIÓN DE RED CEFIRE ||
[RED] Eliminando interfaz dummy 'cefire-dummy'...
[RED] Destruyendo bridge 'LabCEFIRE'...
✓ Red eliminada con éxito.
```

- **Comandos clave:** Hace el proceso inverso al paso 1. Usa ***ip link set LabCEFIRE down*** para apagar la red y ***ip link delete LabCEFIRE*** para eliminarla por completo del Kernel de tu ordenador, liberando la tabla de enrutamiento.

8. PROBLEMAS COMUNES

Si en el mundo real algo falla durante el ***02-deploy-infra.sh***, revisa estos tres sospechosos habituales:

1. **Virtualización por Hardware desactivada:** Si Vagrant lanza un error al intentar arrancar la máquina, es posible que tu placa base tenga desactivado el soporte **VT-x** (Intel) o **AMD-V** (AMD). Tendrás que habilitarlo en la BIOS/UEFI.
2. **Conflictos con la red Bridge:** Si el host no te deja crear la red **LabCE FIRE**, asegúrate de que no tienes otro hipervisor (como Hyper-V en Windows o Docker de red) bloqueando el segmento de red **172.28.0.0/24**.
3. **Falta de memoria RAM:** Si tu PC físico tiene solo 4GB o 8GB de RAM y tienes muchas pestañas de Chrome abiertas, VirtualBox podría fallar al intentar reservar el Gigabyte de memoria que le pedimos para el webserver. Cierra programas e inténtalo con un ***vagrant reload***.